UPTD BALAI LATIHAN KERJA (BLK) PROVINSI GORONTALO

Pelatihan Berbasis Kompetensi

JUNIOR WEB DEVELOPER

MODUL KERJA 5F

Transaksi, Keamanan, dan Notifikasi

Bagian B2: PHP, Berkas JSON, Composer (Jalur Alternatif Tanpa Basis Data)

Batch #5 · Kelas: Junior Web Developer

Instruktur: Rahmat S.A.F. Badaru

2026

Daftar Isi

- A. Identitas Modul
- B. Pemetaan Unit Kompetensi dan Kriteria Capaian
- C. Tujuan Pembelajaran
- D. Petunjuk Belajar
- Kegiatan Belajar 5F.1: Keranjang Belanja dengan localStorage (Hari 16)
- Kegiatan Belajar 5F.2: Endpoint Checkout dan Penguncian Berkas (Hari 17)
- Kegiatan Belajar 5F.3: Composer dan FPDF — Struk PDF (Hari 18)
- Kegiatan Belajar 5F.4: Notifikasi WhatsApp dan Pengujian Akhir (Hari 19)
- E. Lembar Kerja Pendamping
- F. Lembar Asesmen Modul 5F
- G. Glosarium
- H. Referensi
- Lampiran A: Kode Acuan Lengkap
- Lampiran B: Alur Checkout Menyeluruh (Versi Flat-File)

A. Identitas Modul

Komponen	**Keterangan**
Nama modul	Transaksi, Keamanan, dan Notifikasi
Bagian	B2: PHP, Berkas JSON, Composer — jalur alternatif Bagian B, berdampingan (bukan pengganti)
Durasi	32 JP (1 JP = 45 menit), 4 hari × 8 JP, Hari 16-19
Prasyarat	Modul 4F selesai: `api/produk.php` berjalan, folder `data/` terproteksi `.htaccess`, `api/lib/json_store.php` (termasuk `bukaJsonTerkunci`/`tutupJsonTerkunci`) sudah ada, `data/transaksi.json` dan `data/pengguna.json` sudah dibuat (berisi `[]`)
Luaran	(1) Keranjang belanja berfungsi penuh di Kopi Nusantara; (2) checkout tersambung ke `data/transaksi.json` dengan validasi harga dan stok di server, aman dari *race*

Komponen	Keterangan
	condition; (3) struk PDF bisa dilihat/diunduh; (4) tautan konfirmasi WhatsApp ke admin; (5) seluruh alur direplikasi di proyek tema sendiri peserta
Perangkat	Komputer/laptop, Laragon (Apache + PHP 8.5 aktif — **tidak perlu MariaDB**), koneksi internet (untuk memasang Composer)
Perangkat lunak	VS Code, browser dengan DevTools, Composer (dipasang di Hari 17)

Gambaran modul. Modul 4F membangun fondasi yang hanya **membaca** data. Modul 5F adalah inti bisnis sesungguhnya: pelanggan memilih barang, memesan, dan pesanan itu **benar-benar tersimpan** ke `data/transaksi.json` — dengan dua aturan yang tidak boleh dilanggar: **server tidak pernah memercayai harga yang dikirim klien**, dan **penulisan ke berkas data harus terkunci dengan benar** agar tidak ada dua pesanan yang saling menimpa.

Hari	Fokus	Luaran
15	Keranjang belanja (`localStorage`)	Tambah/kurang/hapus item, badge jumlah
16	Endpoint checkout + penguncian berkas	Pesanan tersimpan aman di `transaksi.json`
17	Composer + FPDF	Struk PDF bisa dilihat/diunduh
18	Tautan WhatsApp + pengujian akhir	Alur checkout lengkap, Asesmen Modul 5F

B. Pemetaan Unit Kompetensi dan Kriteria Capaian

Modul ini melanjutkan empat UK yang sama persis dengan Modul 5 versi basis data — pemetaan JP-nya **identik**, karena Bagian B2 berdampingan dengan Bagian B memakai jatah JP yang sama.

UK	Kode Unit	Nama Unit Kompetensi	JP di Modul 5F
1.1	J.620100.005.02	Mengimplementasikan user interface	4
1.4	J.620100.016.01	Menulis kode dengan prinsip sesuai guidelines dan best practices	2
1.5	J.620100.017.02	Mengimplementasikan pemrograman terstruktur	18
1.6	J.620100.019.02	Menggunakan library atau komponen pre-existing	8
		Jumlah	**32**

Kode	Kriteria capaian	Konteks di Modul 5F	Dilatihkan di
1.1-K3	Mengimplementasikan rancangan UI menjadi proses sungguhan	Halaman keranjang dan konfirmasi checkout	KB 5F.1, 5F.2
1.4-K1	Source code sesuai guidelines dan best practices	Harga **selalu** divalidasi ulang di server; penguncian berkas dipakai konsisten	KB 5F.2

Kode	Kriteria capaian	Konteks di Modul 5F	Dilatihkan di
1.5-K1	Program baca tulis sederhana	Checkout membaca **dan** menulis dua berkas sekaligus	KB 5F.2, 5F.3
1.5-K2	Percabangan dan perulangan	Validasi tiap item keranjang, pencarian produk dalam array	KB 5F.1, 5F.2
1.5-K3	Prosedur dan fungsi	Fungsi pengelola keranjang, fungsi pencari produk/transaksi	KB 5F.1, 5F.3
1.5-K4	Array	Struktur data keranjang, array bersarang `items` pada transaksi	KB 5F.1, 5F.2
1.5-K5	Program mengakses data dari media penyimpanan	**Menulis** ke `transaksi.json` dan `produk.json` dengan penguncian penuh	KB 5F.2
1.5-K6	Memperbaiki permasalahan pada program	Menangani stok tidak cukup tanpa merusak data; pengujian menyeluruh	KB 5F.2, 5F.4
1.6-K1	Memilih library yang sesuai	Memilih FPDF untuk kebutuhan cetak struk	KB 5F.3
1.6-K2	Memasang library memakai Composer	Sama seperti Bagian B — Composer tidak bergantung pada jenis penyimpanan data	KB 5F.3
1.6-K3	Memahami lisensi dan pembaruan versi	Lisensi FPDF, `composer.json`/`composer.lock`	KB 5F.3

C. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, Anda mampu:

1. Menyimpan dan mengelola data keranjang belanja di `localStorage`.
2. Menjelaskan mengapa harga yang dikirim dari klien tidak boleh dipercaya, dan menerapkan validasi ulang di server.
3. Memakai `bukaJsonTerkunci()`/`tutupJsonTerkunci()` untuk operasi tulis yang benar-benar tidak boleh tertimpa proses lain.
4. Menerapkan pola "validasi semua dulu, baru tulis semua" sebagai pengganti transaksi basis data.
5. Memasang pustaka pihak ketiga memakai Composer dan membuat dokumen PDF memakai FPDF, membaca data dari berkas JSON.
6. Membuat tautan WhatsApp *click-to-chat* berisi ringkasan pesanan.
7. Menguji alur bisnis menyeluruh dari ujung ke ujung.

D. Petunjuk Belajar

- Modul ini membangun **satu alur yang saling menyambung**: keranjang (Hari 15) memberi data ke checkout (Hari 16), checkout menulis ke `transaksi.json` yang dibaca struk (Hari 17), struk tampil di halaman yang juga memuat tautan WhatsApp (Hari 18).
- **Uji setiap langkah kecil.** Jangan menulis seluruh `api/checkout.php` sekaligus.
- Tautan "Lihat/Unduh Struk" dan "Kirim via WhatsApp" akan **muncul** sejak Hari 16, tapi baru berfungsi penuh di Hari 17 dan 18 — ini disengaja.
- Kode acuan lengkap ada di **Lampiran A**.

Kegiatan Belajar 5F.1: Keranjang Belanja dengan localStorage

Hari 16 · 8 JP · UK 1.1 dan 1.5

Tujuan KB 5F.1. Setelah mengikuti KB ini, Anda mampu melengkapi `produk.json` dengan pengenal numerik, menyimpan data di `localStorage`, membangun fungsi pengelola keranjang, dan menampilkan isi keranjang beserta totalnya.

Kegiatan	**JP**
Melengkapi `produk.json` dengan `id` numerik	1
Konsep `localStorage` dan struktur data keranjang	1
Praktik: fungsi pengelola keranjang (`keranjang.js`)	3
Praktik: tombol "+ Keranjang" dan halaman keranjang	2
Cek kemampuan	1
Jumlah	**8**

Praktik 1: Melengkapi `produk.json` dengan Pengenal Numerik

Rancangan `transaksi.json` dari Modul 4F (Lampiran B) memakai `produk_id` berupa **angka** untuk menandai produk mana yang dipesan — tapi `data/produk.json` sejauh ini hanya mengenal `slug` (teks) sebagai pengenal, tanpa angka `id`. Sebelum keranjang dan checkout bisa dibangun, lengkapi dulu `data/produk.json`: buka di VS Code, tambahkan kolom `"id"` pada **setiap** produk, dimulai dari `1`:

```
{
  "site": { "nama": "Kopi Nusantara", "email": "halo@kopinusantara.test" },
  "menu": [ { "id": "beranda", "label": "Beranda" } ],
  "produk": [
    { "id": 1, "slug": "toraja", "nama": "Kopi Toraja", "asal": "...", "kategori": "arabika",
"harga": 85000, "stok": 20, "deskripsi": "...", "foto": "...", "alt": "..." },
    { "id": 2, "slug": "gayo", "nama": "Kopi Gayo", "...": "..." }
  ]
}
```

Lanjutkan sampai seluruh produk (enam produk sejak Bagian A) memiliki `id` unik 1 sampai 6. Simpan, uji ulang `api/produk.php` — pastikan kolom `id` ikut tampil di hasilnya.

Materi 1: *localStorage* dan Struktur Data Keranjang

`localStorage` adalah tempat penyimpanan kecil milik browser, khusus satu alamat website, yang tetap ada meski tab ditutup. Nilai yang disimpan harus berupa teks, sehingga menyimpan array/objek butuh `JSON.stringify()`/`JSON.parse()` — pola yang sama persis seperti membaca JSON dari server di Modul 3 dan 4F, hanya sumbernya kini browser sendiri.

Keranjang disimpan sebagai satu array, satu objek per produk:

```
[
  { produk_id: 1, slug: "toraja", nama: "Kopi Toraja", harga: 85000, jumlah: 2 }
]
```

Catatan penting: `harga` di sini **hanya untuk tampilan**. Server (Hari 16) akan mengabaikan nilai ini sepenuhnya dan mengambil harga sesungguhnya langsung dari `data/produk.json`. `localStorage` tidak pernah menjadi sumber kebenaran soal harga.

Praktik 2: Fungsi Pengelola Keranjang

Buat `js/keranjang.js`:

```
// keranjang.js: mengelola keranjang belanja di localStorage.
// CATATAN: harga di sini hanya untuk tampilan. Server (checkout) yang
// menentukan harga sesungguhnya -- localStorage tidak dipercaya.

const KUNCI_KERANJANG = "kopi-nusantara-keranjang";

function ambilKeranjang() {
  const data = localStorage.getItem(KUNCI_KERANJANG);
  return data ? JSON.parse(data) : [];
}

function simpanKeranjang(keranjang) {
  localStorage.setItem(KUNCI_KERANJANG, JSON.stringify(keranjang));
}

function tambahKeKeranjang(produk) {
  const keranjang = ambilKeranjang();
  const ada = keranjang.find(function (item) {
    return item.produk_id === produk.produk_id;
  });

  if (ada) {
    ada.jumlah += 1;
  } else {
    keranjang.push({
      produk_id: produk.produk_id,
      slug: produk.slug,
      nama: produk.nama,
      harga: produk.harga,
      jumlah: 1,
    });
  }

  simpanKeranjang(keranjang);
  perbaruiBadgeKeranjang();
}

function ubahJumlah(produkId, delta) {
  const keranjang = ambilKeranjang();
  const item = keranjang.find(function (i) {
    return i.produk_id === produkId;
  });
  if (!item) return;

  item.jumlah += delta;

  const keranjangBaru = item.jumlah <= 0
    ? keranjang.filter(function (i) { return i.produk_id !== produkId; })
    : keranjang;

  simpanKeranjang(keranjangBaru);
  renderKeranjang();
  perbaruiBadgeKeranjang();
}
```

```
function hapusDariKeranjang(produkId) {
  const keranjang = ambilKeranjang().filter(function (i) {
    return i.produk_id !== produkId;
  });
  simpanKeranjang(keranjang);
  renderKeranjang();
  perbaruiBadgeKeranjang();
}

function hitungTotalKeranjang() {
  return ambilKeranjang().reduce(function (total, item) {
    return total + item.harga * item.jumlah;
  }, 0);
}

function perbaruiBadgeKeranjang() {
  const badge = document.getElementById("badge-keranjang");
  if (!badge) return;

  const jumlahItem = ambilKeranjang().reduce(function (t, i) {
    return t + i.jumlah;
  }, 0);

  badge.textContent = jumlahItem;
  badge.hidden = jumlahItem === 0;
}
```

Jangan lupa memuat berkas ini. Buka `index.html`, cari baris `<script src="js/app.js"></script>` di akhir `<body>`, tambahkan satu baris **setelahnya**:

```
<script src="js/app.js"></script>
<script src="js/keranjang.js"></script>
```

Uji dulu lewat Console (F12): panggil `tambahKeKeranjang({produk_id:1, slug:"toraja", nama:"Kopi Toraja", harga:85000})` beberapa kali, lalu `ambilKeranjang()` dan `hitungTotalKeranjang()` untuk memastikan datanya benar.

Praktik 3: Menyambungkan ke Tampilan

Langkah 1 — tombol di tiap kartu. Buka `js/app.js`, ubah `kartuProduk(p)`:

```
function kartuProduk(p) {
  return `
    <article class="card">
      <img src="${esc(p.foto)}" alt="${esc(p.alt)}" class="product-photo">
      <h3>${esc(p.nama)}</h3>
      <p class="meta">${esc(p.asal)} &middot; ${esc(p.kategori)}</p>
      <p>${esc(p.deskripsi)}</p>
      <p class="price">${rupiah(p.harga)}</p>
      <button class="btn btn-keranjang"
              data-id="${p.id}" data-slug="${esc(p.slug)}"
              data-nama="${esc(p.nama)}" data-harga="${p.harga}">
        + Keranjang
      </button>
    </article>
  `;
}
```

`p.id` kini tersedia karena Praktik 1 menambahkannya ke `data/produk.json`.

Langkah 2 — *event delegation* untuk tombol. Tambahkan di `app.js`:

```
function pasangTombolKeranjang() {
  daftarEl.addEventListener("click", function (e) {
    const tombol = e.target.closest(".btn-keranjang");
    if (!tombol) return;

    tambahKeKeranjang({
      produk_id: Number(tombol.dataset.id),
      slug: tombol.dataset.slug,
      nama: tombol.dataset.nama,
      harga: Number(tombol.dataset.harga),
    });

    tombol.textContent = "\u2713 Ditambahkan";
    setTimeout(function () {
      tombol.textContent = "+ Keranjang";
    }, 800);
  });
}
```

Panggil `pasangTombolKeranjang();` di dalam `mulai()`.

Langkah 3 — tombol keranjang di header. Tambahkan di dalam `<header class="site-header">`:

```
<button id="tombol-keranjang-header" class="cart-icon" onclick="location.hash='#keranjang'">
  🛒 <span id="badge-keranjang" class="badge" hidden>0</span>
</button>
```

Langkah 4 — bagian keranjang. Tambahkan `<section>` baru di dalam `<main class="site-content">`:

```
<section id="keranjang">
  <h2>Keranjang Belanja</h2>
  <div id="isi-keranjang"></div>
  <p class="total-keranjang">Total: <span id="total-keranjang">Rp 0</span></p>
</section>
```

Langkah 5 — fungsi `renderKeranjang()`. Tambahkan di `keranjang.js`:

```
function renderKeranjang() {
  const isiEl = document.getElementById("isi-keranjang");
  const totalEl = document.getElementById("total-keranjang");
  const keranjang = ambilKeranjang();

  if (keranjang.length === 0) {
    isiEl.innerHTML = "<p>Keranjang masih kosong.</p>";
    totalEl.textContent = rupiah(0);
    return;
  }

  isiEl.innerHTML = keranjang.map(function (item) {
    return `
      <div class="baris-keranjang">
        <span>${esc(item.nama)}</span>
        <button data-aksi="kurang" data-id="${item.produk_id}">-</button>
        <span>${item.jumlah}</span>
        <button data-aksi="tambah" data-id="${item.produk_id}">+</button>
        <span>${rupiah(item.harga * item.jumlah)}</span>
```

```
        <button data-aksi="hapus" data-id="${item.produk_id}">Hapus</button>
      </div>
    `;
  }).join("");

  totalEl.textContent = rupiah(hitungTotalKeranjang());
}

function pasangAksiKeranjang() {
  document.getElementById("isi-keranjang").addEventListener("click", function (e) {
    const tombol = e.target.closest("[data-aksi]");
    if (!tombol) return;

    const id = Number(tombol.dataset.id);
    const aksi = tombol.dataset.aksi;

    if (aksi === "tambah") ubahJumlah(id, 1);
    if (aksi === "kurang") ubahJumlah(id, -1);
    if (aksi === "hapus") hapusDariKeranjang(id);
  });
}
```

Panggil `renderKeranjang();`, `pasangAksiKeranjang();`, dan `perbaruiBadgeKeranjang();` di dalam `mulai()` juga.

Latihan 1

1. Uji: tambahkan 3 produk berbeda, ubah jumlah salah satunya jadi 0 lewat tombol "-" berulang kali — pastikan baris itu hilang otomatis.
2. Segarkan halaman (`F5`) setelah mengisi keranjang — pastikan isinya tidak hilang.
3. Buka DevTools, tab **Application/Storage**, cari kunci `kopi-nusantara-keranjang`, amati isinya.
4. *Tantangan:* tambahkan tombol "Kosongkan Keranjang".

Rangkuman KB 5F.1

- `data/produk.json` kini memiliki `id` numerik, melengkapi rancangan `transaksi.json` dari Modul 4F.
- `localStorage` menyimpan data keranjang sebagai teks; harga yang tersimpan di sana hanya untuk tampilan.
- Pola keranjang (fungsi, event delegation, render ulang) sama sekali tidak bergantung pada jenis backend — ini seluruhnya kode sisi klien.

Cek Kemampuan KB 5F.1

- ☐ Seluruh produk di `data/produk.json` memiliki `id` numerik unik.
- ☐ Tombol "+ Keranjang" dan halaman keranjang berfungsi penuh.
- ☐ Badge jumlah di header diperbarui otomatis.
- ☐ Isi keranjang tetap ada setelah halaman disegarkan.

Kesalahan umum: lupa menambahkan `id` ke salah satu produk (atau id-nya kembar); lupa `<script src="js/keranjang.js">`; `produk_id` tertukar dengan `slug`.